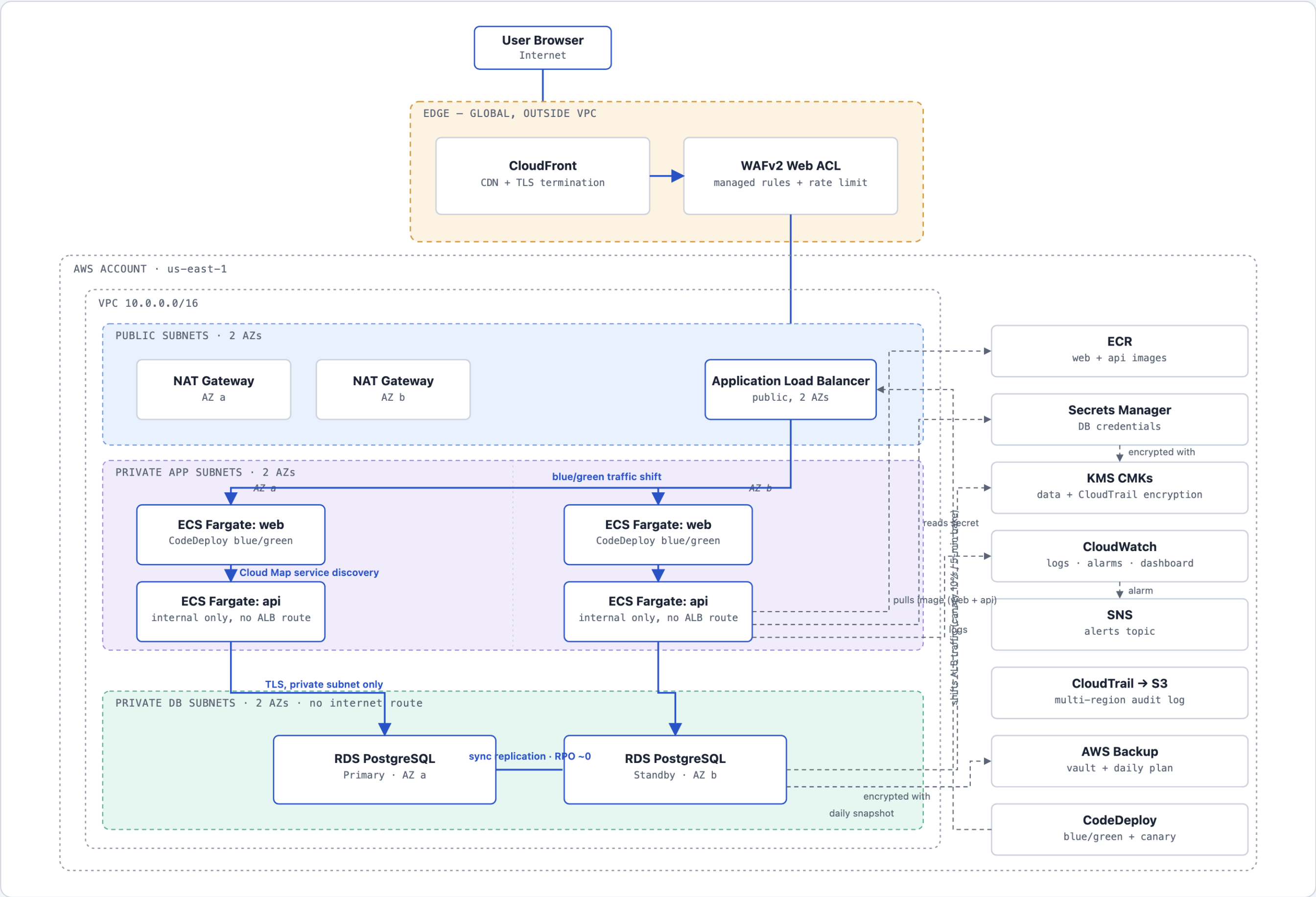


Secure 3-Tier AWS Architecture

Production topology for **toptal / prod** — a server-rendered Express + Pug web tier, an internal-only API tier, and a Multi-AZ RDS database, fronted by CloudFront + WAF and deployed through CodeDeploy blue/green.



Request path (solid, blue) runs User → CloudFront → WAF → ALB → web (ECS Fargate) → api (ECS Fargate, internal-only) → RDS Multi-AZ. Dashed grey lines are supporting control-plane traffic: image pulls, secret reads, log shipping, encryption, backups, and the CodeDeploy traffic shift. Colored zones mark trust boundaries — edge, public subnet, private app subnet, private DB subnet.

Tiers

TIER	COMPUTE	REACHABILITY	NOTES
Web	ECS Fargate · 2+ tasks · 2 AZs	Public via CloudFront → WAF → ALB	CodeDeploy blue/green (10% canary, 5-min bake, alarm-triggered rollback)
API	ECS Fargate · 2+ tasks · 2 AZs	Private only — Cloud Map service discovery, no ALB route	ECS-native rolling deploy (100% min healthy = zero downtime)
Database	RDS PostgreSQL · Multi-AZ	Private DB subnets only, no internet route, SG allows API tier only	Automated backups + independent AWS Backup daily plan

Why these choices

CloudFront in front of an ALB, not S3

The app is server-rendered (Express + Pug), so the origin has to be compute. CloudFront still buys CDN caching for static assets, edge TLS, and a WAF attachment point.

Fargate over EC2/EKS

Two small stateless HTTP services don't need a cluster control plane or node patching — Kubernetes' scheduling model would be pure overhead here.

API has no ALB route

It's only ever called by the web tier over internal service-discovery DNS. Exposing it publicly would be unused, unnecessary attack surface.

Blue/green for web, rolling for API

CodeDeploy traffic shifting needs two target groups behind a load balancer. Web has one (the public ALB); API is internal-only, so it uses ECS's native rolling deploy instead.

Multi-AZ RDS + per-AZ NAT

Both exist to survive an AZ failure. A shared NAT Gateway or single-AZ database are the two most common "looks production-ready but isn't" mistakes in a 3-tier design.

Secrets never touch plaintext

The DB password is generated by `random_password` at apply time and read by ECS via the Secrets Manager ARN — never a Terraform variable, env var, or git-committed value.